

# Evaluating ML models.

## *Measure. Compare. Improve.*

Count the right errors. Compare models fairly.

Turn evaluation into a reasoned next step.

🕒 18-22 min core reading · 20-30 min practice · 12 short sections

---

A model can look impressive on one number and still make the wrong mistakes. Evaluation connects predictions to a clear task, an honest comparison, and a decision about what to improve next.

### WHAT YOU WILL UNDERSTAND

**The main question:** does this model help with the problem we actually care about, on examples it did not learn from?

**The practical path:** count errors, choose meaningful metrics, compare against a baseline, protect the test set, and investigate recurring failures.

**Reading path:** concise explanations, three diagrams, an interactive evaluation workbench, and a Python workbook. Expand the deeper notes for extra detail.

# 1. Define success before comparing scores

For our movie classifier, **positive means an observed thumbs-up**. One example is a viewer-movie event; the prediction estimates whether that viewer will like that movie. This is different from predicting a star rating or ranking a whole catalog.

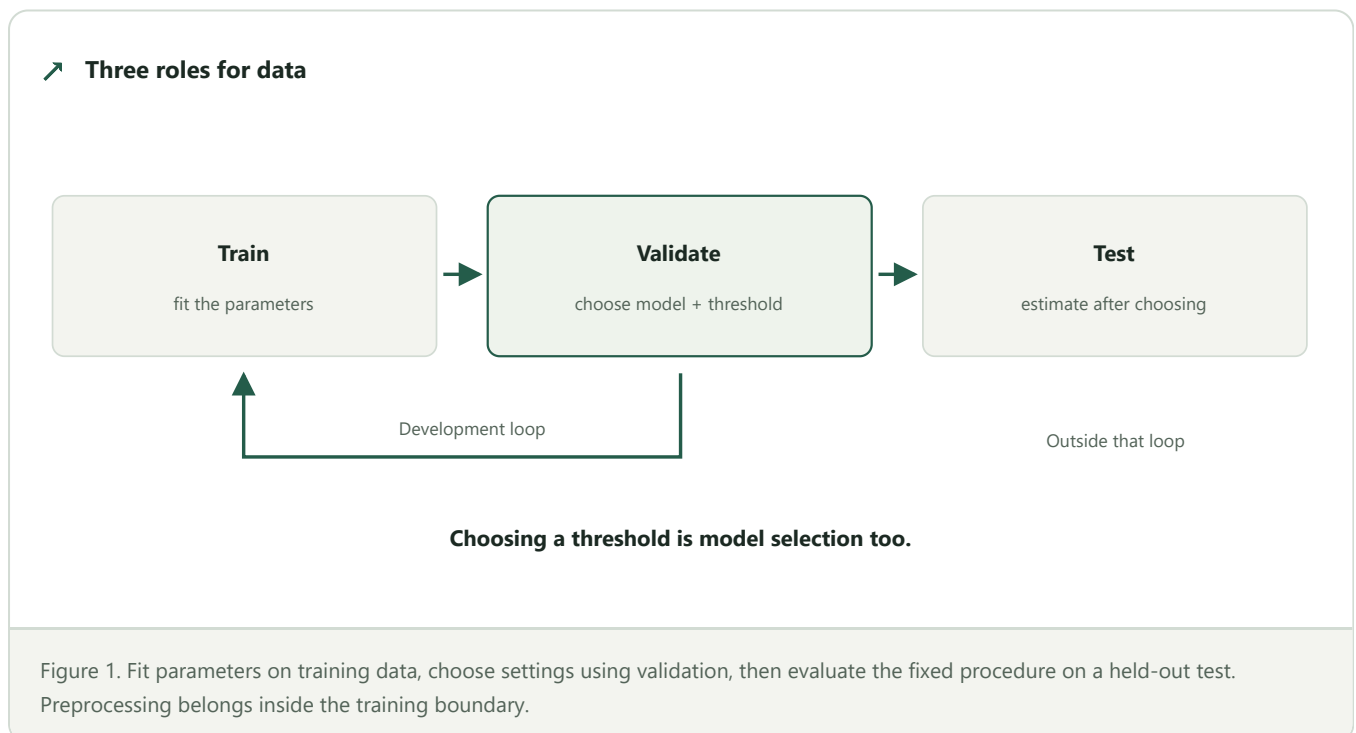
Decide which errors matter. An unwanted suggestion can waste a viewer's attention; a missed like can hide a film they would enjoy. Choose a primary comparison rule and useful secondary measures before searching for a winning model. Hard requirements, such as a 120-minute runtime limit, still need their own checks.

Always include a **baseline**. In this chapter's 12 validation events, only three are likes. Predicting thumbs-down for everyone gets nine correct: **75% accuracy, but zero likes found**. A headline accuracy score needs context.

Our explorer uses a new synthetic snapshot: two sets of **authored, frozen probabilities**, labeled A and B, plus observed labels. It calculates real metrics but trains no model. A separate authored training summary, 6 likes among 24 events, supplies the constant baseline probability  $6/24 = 0.25$ . These are teaching fixtures, not results from Chapter 13's fitted models or a real recommendation service.

## 2. Protect the boundary between choosing and testing

Use **training data** to fit model parameters. Use **validation data** to choose model families, features, regularization, stopping points, or thresholds. Use a **test set** to assess the procedure after those choices are fixed. A threshold is part of the delivered system, even if changing it does not retrain the model.



The split should resemble deployment. For unseen viewers, keep a viewer's events together; for future events, respect time. Stratification preserves class proportions but does not, by itself, prevent viewer overlap or future information from leaking backward.

Fit learned imputation, scaling, PCA, and feature selection inside the training boundary. Apply the saved transformation to held-out rows. This remains necessary even when the preprocessing is unsupervised. See [scikit-learn's data-leakage guide](#).

---

▼ Go deeper: [cross-validation and nested selection](#)

---

In **k-fold cross-validation**, divide development data into k folds. For each round, fit a fresh pipeline on the other folds and evaluate on the held-out fold. A three-fold schedule is: train on B+C, validate on A; train on A+C, validate on B; train on A+B, validate on C. Fit preprocessing again inside each round. Use group-aware or time-aware folds when the prediction setting requires them.

Fold scores show sensitivity to the chosen data. Their simple mean weights folds equally; pooling errors weights examples equally, which differs when fold sizes differ. Fold standard deviation is not automatically a confidence interval because the fitted models share training data.

If the same cross-validation scores select the best configuration, reporting that best score alone can be optimistic. An untouched test set or **nested cross-validation** separates selection from assessment: inner folds choose settings; outer folds evaluate that selection process. See the primary [cross-validation guide](#) and [nested-CV example](#). The current workbook compares supplied scores; it does not fit cross-validation models.

---

## 3. Read the confusion matrix before the headline

Every binary decision belongs to one of four outcomes. A **true positive (TP)** finds a like; a **false positive (FP)** predicts a like for an observed dislike. A **true negative (TN)** correctly predicts a dislike; a **false negative (FN)** misses a like. "True" means agreement with the recorded label, whose quality still matters.

### ➤ Four outcomes behind a single accuracy score

|                  | Predict thumbs-up              | Predict thumbs-down         |
|------------------|--------------------------------|-----------------------------|
| Observed like    | 3<br>TP · found likes          | 0<br>FN · missed likes      |
| Observed dislike | 2<br>FP · unwanted suggestions | 7<br>TN · correct negatives |

Accuracy =  $(3 + 7) / 12$ . Precision =  $3 / 5$ . Recall =  $3 / 3$ .

Figure 2. Candidate A at threshold 0.50 on 12 validation events. Rows are observed labels; columns are predictions. Positive means a thumbs-up.

For Candidate A at threshold 0.50, the counts are TP = 3, FP = 2, TN = 7, FN = 0. Different metrics ask different questions:

| Metric    | Calculation           | Result            | Question answered                              |
|-----------|-----------------------|-------------------|------------------------------------------------|
| Accuracy  | $(TP + TN)/n$         | $10/12 = 83.33\%$ | What fraction of all decisions is correct?     |
| Precision | $TP/(TP + FP)$        | $3/5 = 60\%$      | Of predicted likes, how many are actual likes? |
| Recall    | $TP/(TP + FN)$        | $3/3 = 100\%$     | Of actual likes, how many did we find?         |
| F1        | $2TP/(2TP + FP + FN)$ | $6/8 = 0.75$      | How do precision and recall combine?           |

F1 is their harmonic mean, not their ordinary average. It ignores true negatives and does not encode every possible error cost. See Google's [classification-metrics](#) explanation.

### ▼ Go deeper: undefined values, class balance, and averaging

If no events are predicted positive, precision divides by zero. We show **Undefined** and write JSON null, rather than pretending it is perfect precision. Recall is undefined if there are no actual positives. F1 is undefined when  $2TP + FP + FN$  is zero. A library may choose a reporting convention; know which one it uses.

**Specificity** is  $TN/(TN+FP)$ , the fraction of actual negatives recognized. **Balanced accuracy** averages recall and specificity, giving both classes equal weight regardless of their frequency; it is undefined here if a class is absent. For multiple classes, **macro** averaging treats class-level scores equally, **weighted** averaging weights them by class support, and **micro** averaging pools counts before computing the metric. State both the averaging rule and the class counts. See the [classification-report](#) reference.

---

## 4. Choose a threshold for a reason

We predict thumbs-up when  $p \geq t$ . Changing  $t$  changes decisions, not the supplied probabilities. Raising it shrinks the set predicted positive: false positives cannot increase and recall cannot increase on a fixed dataset.

**Precision need not improve at every step**, because both its numerator and denominator change.

One explicit comparison rule is weighted error cost:

$$C = c_{FP} FP + c_{FN} FN.$$

Our default teaching assumption gives each unwanted suggestion cost 1 and each missed like cost 3. Candidate A at threshold **0.60** has one FP and no FN, so cost is 1. Candidate B at **0.70** has no FP and one FN, so cost is 3. Both have **91.67% accuracy**. Reverse the costs and B becomes preferable under that rule.

Choose these assumptions from the application's goals, then select a policy using validation data. Do not claim that 0.60 is universally optimal; it is best among this workbook's preset choices under one authored scenario. See scikit-learn's [threshold-selection guide](#).

---

## 5. Separate ranking quality from one operating point

A **ROC curve** sweeps thresholds and plots recall against the false positive rate,  $FP/(FP + TN)$ . One threshold is one operating point. **ROC-AUC** summarizes the area under the whole curve, rather than the decisions at the currently selected threshold.

For a positive-negative pair, AUC gives credit 1 when the positive has a higher score, 0 when lower, and 1/2 for a tie. Candidate A orders **25 of 27** positive-negative pairs correctly: AUC is  $25/27 \approx 0.9259$ . Its threshold can move while that ordering stays fixed. This is not 92.59% accuracy at a chosen cutoff.

A **precision-recall curve** instead shows precision against recall as the threshold changes. It focuses attention on finding positives without filling the suggested set with negatives, which is useful when positives are rare. Class prevalence affects interpretation and comparison. See Google's [ROC and AUC guide](#).

---

### ▼ Go deeper: AUC ties, average precision, and top-K results

A constant score gives ROC-AUC 0.5 when both classes are present, because every positive-negative pair ties. Our ROC implementation processes equal scores together; ordering tied rows arbitrarily must not change the result. ROC-AUC is undefined with only one observed class. The curve's axes are rates, not probability calibration or business value.

**Average precision (AP)** summarizes precision over increases in recall. It is not generally identical to a trapezoidal area under a precision-recall curve. For ranked movie lists, **precision@K** asks how many of the top K results are relevant; **recall@K** asks how much known relevant material those results recover. A relevance label and a candidate set must be defined first. A high whole-catalog AUC does not guarantee a useful top-five list. We develop retrieval metrics in Part 5; this explorer implements ROC-AUC, not AP or top-K ranking. See the primary [AP reference](#) and Stanford's [ranked-retrieval explanation](#).

## 6. Match the metric to numerical predictions

For a star-rating regressor, Chapter 13's **MAE** measures average absolute error; **RMSE** takes the square root of average squared error, emphasizing large misses. Both use rating units. Inspect residuals too: similar overall errors can hide consistent underprediction for one type of movie.

**R-squared** compares squared error with predicting the mean target on the evaluation set:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

For observed ratings `[1, 3, 5]`, the mean is 3 and the denominator is 8. Predicting `[3, 3, 3]` gives  $R^2 = 0$ ; predicting `[5, 5, 5]` gives  $1 - 20/8 = -1.5$ . **R-squared can be negative**; it is not "percent accurate." With a constant target, this denominator is zero. See the [R-squared reference](#) for library conventions.

That evaluation-set mean defines the R-squared formula; a deployed constant baseline must still be fitted from training data. Compare models on the same examples, target units, and metric definition. Do not compare regression RMSE directly with classification F1.

## 7. Check probabilities and the strength of the evidence

**Calibration** asks whether predictions near 0.8 correspond to likes about 80% of the time across suitable cases. A **reliability diagram** compares average prediction with observed positive frequency in probability bins. A sigmoid or a good ranking does not automatically give calibrated probabilities.

**Log loss** penalizes assigning low probability to the observed outcome. The binary **Brier score** averages  $(p - y)^2$ . Smaller is better for both, but neither is a pure calibration measure: they also reflect other aspects of predictive quality. See scikit-learn's [calibration guide](#).

Our candidates have the same AUC, approximately 0.9259, but different log losses: A is 0.4485; B is 0.3681. The supplied probability values differ even when the overall ranking score agrees. These 12 synthetic events cannot establish reliable real-world calibration or superiority.

### ▼ Go deeper: uncertainty and comparing the same cases

Report sample sizes alongside metrics. One additional mistake changes accuracy by  $1/12$ , about 8.33 percentage points, in this validation set. An apparent improvement on very few events can be fragile. Compare candidates on the same examples; otherwise, differences in difficulty can masquerade as model improvements.

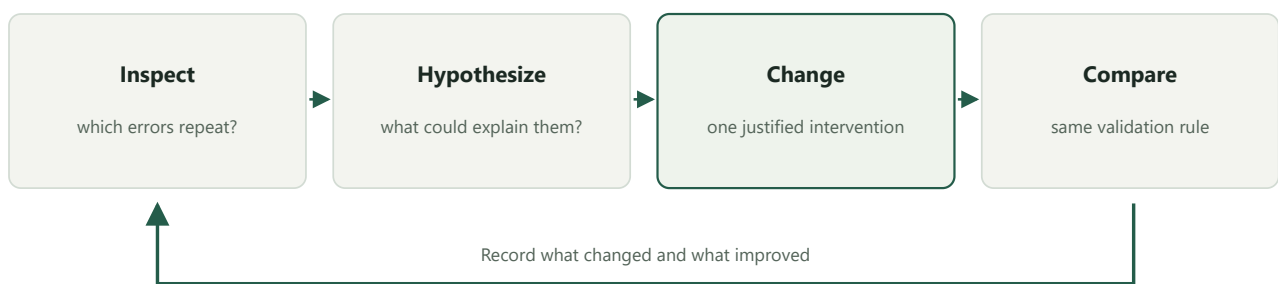
When estimating uncertainty, preserve the unit of dependence: repeated events from one viewer should not automatically count as independent people. A bootstrap comparison can resample viewers and compute the paired metric difference for each resample. Time dependence needs a suitable time-aware method. These intervals assess sampling variability under assumptions, not protection from leakage or distribution shift; see Chapter 7's uncertainty discussion.

The workbook clips probabilities to  $[10^{-15}, 1 - 10^{-15}]$  only when computing log loss, so endpoint mistakes remain finite in JSON. The mathematical loss for a confidently impossible observed outcome is infinite. Brier scores, decisions, and AUC use the original scores. No confidence interval or fitted calibrator is produced by this demo.

## 8. Improve the cause of errors, not just the score

Inspect examples and meaningful **slices** before choosing an intervention. At threshold 0.50, Candidate A gets four of six short-history events correct and all six long-history events correct. That suggests investigating the short-history cases; it does not prove history length caused the errors. Each slice is tiny and has a different positive rate.

### ↗ Improve one explanation of the errors at a time



**More complexity is one possible change, not the default answer.**

Figure 3. Inspect representative errors, propose a cause, change the system, and compare on the same validation protocol. Keep the final test out of this loop.

| Observed pattern                            | Investigate                                            | Possible response                                  |
|---------------------------------------------|--------------------------------------------------------|----------------------------------------------------|
| Training and validation both perform poorly | Missing signal, wrong target, excessive constraints    | Improve features or labels; reconsider capacity    |
| Training is strong, validation much worse   | Leakage, excessive flexibility, unrepresentative split | Fix the split; regularize; collect relevant data   |
| Failures concentrate in one slice           | Label quality, coverage, missing context               | Inspect cases; improve that data or use a fallback |
| Ranking is useful but decisions are costly  | Threshold and error assumptions                        | Tune the policy on validation                      |

These are hypotheses, not diagnoses from a score alone. Record the data version, feature pipeline, candidate settings, split, metric, and result. Change something for a stated reason and compare again. **An ablation** removes or changes one component to test its contribution; it supports a narrower claim than changing everything at once.

#### ▼ Go deeper: learning curves and tuning without chasing noise

A **learning curve** compares training and validation performance as training-set size grows. A **validation curve** changes a setting such as regularization while holding the evaluation protocol fixed. Poor training performance can suggest underfitting; a persistent gap can suggest variance or a data mismatch. These patterns guide investigation, not automatic conclusions. See the [learning-curve guide](#).

**Grid search** tries a preset combination list; **random search** samples settings from specified distributions. More trials also create more opportunities to win through validation noise. Keep a deliberate search budget and an independent final assessment. **Early stopping** uses validation behavior to decide when training ends, so the stopping choice also belongs to selection.

Class weighting or resampling may help some imbalanced tasks, but apply resampling only inside training folds; preserve the held-out population you intend to measure. Do not oversample before splitting, where copies can cross the boundary. If probabilities or the class balance change, revisit calibration and thresholds. See the [primary parameter-search guide](#).

## 9. Explore the evaluation workbench

Start with Candidate A and threshold 0.50. Check the four counts and **83.33% accuracy, 60% precision, 100% recall**. Select the constant baseline and explain its 75% accuracy. With no predicted likes, precision is undefined.

Return to A and raise the threshold to 0.85. Precision becomes 100%, but recall falls to 33.33%. AUC and log loss stay fixed. Compare the short- and long-history slices, then change the cost assumption and inspect which preset configuration wins on validation.

## Same predictions. Different decisions.

Twelve fictional validation events contain observed thumbs-up labels and two frozen sets of invented probabilities. We calculate their metrics here; no model is trained by this explorer.

Default: Candidate A, threshold 0.50, all 12 validation events, FP cost 1 and FN cost 3. TP = 3, FP = 2, TN = 7, FN = 0. Accuracy is 83.33%, precision 60%, recall 100%, and F1 0.7500.

| Prediction set / threshold | Accuracy | Precision | Recall  | Cost |
|----------------------------|----------|-----------|---------|------|
| Candidate A / 0.50         | 83.33%   | 60.00%    | 100.00% | 2    |
| Candidate A / 0.60         | 91.67%   | 75.00%    | 100.00% | 1    |
| Candidate B / 0.70         | 91.67%   | 100.00%   | 66.67%  | 3    |
| Constant baseline / 0.50   | 75.00%   | Undefined | 0.00%   | 9    |

Candidate A has ROC-AUC 0.9259, log loss 0.4485, and Brier score 0.1438. Moving only its threshold changes none of these three values. The fixed validation comparison selects A at 0.60 under the default cost rule. With FP cost 3 and FN cost 1, it selects B at 0.70. These are synthetic selection results, not measured production gains.

The fixed comparison tries A and B at thresholds 0.40, 0.50, 0.60, 0.70, and 0.85. It always uses **all validation events** and minimizes total weighted error cost; exact ties keep the first listed configuration. The manual controls above explore individual policies and slices. A lower total cost on a smaller slice is not automatically better: inspect the event count and cost per event too.

## 10. Run the workbook and practice

Extract its four files together. Python 3.10 or newer is enough; no third-party packages, credentials, or API calls are needed.

```
py evaluate.py
py evaluate.py --candidate baseline
py evaluate.py --threshold 0.85
py evaluate.py --slice short
py evaluate.py --cost unwanted
py -m unittest -v test_evaluate.py
```

Use `python` or `python3` if that is your installed command. The script reads `movie_predictions.json` beside itself and prints JSON, leaving input files unchanged. Invalid data or out-of-range probabilities and thresholds stop with an error and exit code 2.

After fixing the **selection rule and cost assumption**, run `py evaluate.py --evaluate-test`. It selects using validation, then evaluates that selected policy on all eight test events. Manual `--candidate`, `--threshold`, and `--slice` settings affect the exploration report, not which policy gets this final test. The flag encourages a workflow; the local test labels are not secret or access-controlled. Retuning after seeing them would make them development data.

## Exercise 1: explain three different percentages

Using  $TP = 3$ ,  $FP = 2$ ,  $TN = 7$ ,  $FN = 0$ , calculate accuracy, precision, and recall. Why are their denominators different?

### ▼ Reveal the reasoning

Accuracy is  $(3 + 7)/12 = 83.33\%$ , across all events. Precision is  $3/(3 + 2) = 60\%$ , across predicted likes. Recall is  $3/(3 + 0) = 100\%$ , across actual likes. The model finds every recorded like but also suggests two movies whose events were labeled dislikes.

## Exercise 2: choose between equal accuracies

A at threshold 0.60 makes one FP and zero FN. B at 0.70 makes zero FP and one FN. Compare costs when FP costs 1 and FN costs 3; then reverse those weights.

### ▼ Reveal the reasoning

First, A costs 1 and B costs 3, so A wins under that rule. Reversing the weights makes A cost 3 and B cost 1. Both still have 11/12 accuracy. The preferable decision policy depends on which error matters more, not just the number correct.

## Exercise 3: keep test feedback out of selection

In a copy of the JSON, flip the test event E013's observed label from 1 to 0. Run the default command and then `--evaluate-test`. Which outputs should change?

### ▼ Reveal the reasoning

The validation report, comparison, and selected configuration must stay unchanged. Only the final-test report changes: the default selected A/0.60 predicts positive for E013, so one test true positive becomes a false positive. The tests check this boundary. Restore the original snapshot before comparing handbook results; do not tune using the changed test labels.

## 11. Check your understanding

### 01 A constant negative predictor gets 75% accuracy. What else must you inspect?

- A. Nothing; 75% proves the recommender is useful.
- B. Class balance and which likes or dislikes it misses.
- C. Only how quickly the code runs.

**Answer: B.** With three likes and nine dislikes, always predicting negative is 75% accurate but finds none of the likes. Accuracy alone hides that failure.

### 02 $TP = 3$ , $FP = 2$ , and $FN = 0$ . What are precision and recall?

- A. Precision 60%; recall 100%.
- B. Precision 100%; recall 60%.
- C. Both are 83.33%.

**Answer: A.** Precision is  $3/(3+2)$ , while recall is  $3/(3+0)$ . They ask about different denominators.

### 03 Only the decision threshold changes. Which quantity stays fixed on the same events?

- A. The number of false positives.
- B. The number of missed likes.
- C. ROC-AUC and probability-based log loss.

**Answer: C.** Those two measures use the unchanged scores rather than just the decisions at the current cutoff. Counts and decision metrics may change.

### 04 How should preprocessing be handled during cross-validation?

- A. Fit it once on all development rows before making folds.
- B. Fit it independently on each training fold, then transform that fold's held-out rows.
- C. Fit it on the final test set for consistency.

**Answer: B.** Imputation, scaling, and feature selection can learn from data. Fitting them outside the training-fold boundary can leak held-out information.

### 05 After seeing disappointing test results, you tune the threshold on those labels. What changes?

- A. The test remains untouched because the model weights did not change.
- B. The old test score becomes an unbiased estimate of the new threshold.
- C. The test has entered model selection; you need fresh independent evaluation for the revised procedure.

**Answer: C.** Thresholds are part of the delivered policy. Reusing test feedback to choose them makes the test part of development.

## 12. Make a claim your evidence supports

You can now connect **task** → **evaluation split** → **baseline** → **metrics and error costs** → **validation selection** → **final assessment**. Improvement means a useful change under that full procedure, not merely a larger number somewhere in a report.

Offline movie labels also have limits: unobserved feedback is not automatically a dislike, and feedback about shown movies does not reveal how viewers would react to every unseen film. Before deployment, assess coverage, latency, cost, and the actual product outcome. Monitor whether the data and results change over time; later production chapters develop that loop.

### ▼ Quick-reference glossary

| Term               | Meaning here                                                       |
|--------------------|--------------------------------------------------------------------|
| Baseline           | Simple reference system measured on the same task                  |
| Validation / test  | Data for choosing / data for assessment after choosing             |
| Confusion matrix   | Counts of correct and incorrect decisions by observed class        |
| Precision / recall | Correct among predicted positives / found among actual positives   |
| F1                 | Harmonic combination of precision and recall                       |
| Threshold          | Cutoff converting scores into decisions                            |
| ROC-AUC            | Ranking summary across a sweep of thresholds                       |
| Calibration        | Agreement between predicted probabilities and observed frequencies |
| MAE / RMSE         | Absolute-error average / root mean squared error                   |
| R-squared          | Squared-error comparison with an evaluation-set mean reference     |
| Cross-validation   | Repeated fitting and evaluation across held-out folds              |
| Slice / ablation   | A meaningful data subset / a controlled component change           |

**Primary references:** Google's [classification metrics](#) and ROC/AUC; scikit-learn's [cross-validation](#), [leakage prevention](#), [calibration](#), and [learning curves](#). All movies, labels, and candidate scores in the workbook are original synthetic teaching data.

**Next: Chapter 16 - Information retrieval: how search works.** Begin Part 5 by finding relevant material before building embeddings and RAG.